

Lab 4 — Vibe Coding: chat history

Fruit Ninja Lite (SETAPESU26/41_fruit-ninja) • assistant: Claude

Name	SRN
Prajwal G Devadiga	PES1UG24CS590

PROMPT 1

```
@"C:\Users\prajw\Desktop\SE-Labs-PES1UG24CS590\Lab-4\code\main.py"
@"C:\Users\prajw\Desktop\SE-Labs-PES1UG24CS590\Lab-4\code\game\fruit.py"
@"C:\Users\prajw\Desktop\SE-Labs-PES1UG24CS590\Lab-4\code\game\game_engine.py"
Hey, I've got a small Fruit Ninja style game in Pygame that I need to fix up for a lab. I've pasted the three files below.
```

The problem: when I swipe quickly across a fruit the blade goes straight through it and nothing happens -- no slice, no score. But if I move the mouse slowly over the same fruit it cuts fine every time. My guess is it's only looking at wherever the mouse is at that instant instead of the line it travelled along, so a fast swipe just skips over the fruit between frames.

Have a quick read of the code first and tell me if that's actually the cause, then fix it so fast swipes register properly. Also give me some way to test it without playing the game over and over -- something I can just run and see pass or fail.

RESPONSE 1

I'll start by reading the rest of the project to see what's there (the code comments reference a README with tasks).

Diagnosis confirmed -- your read of it is exactly right. Let me make the fix.

Now a runnable test that exercises this headlessly:

Let me verify the tests actually fail against the old point-only behaviour (so they're a real regression guard, not just green):

Your diagnosis is correct. Confirmed in the code:

```
`_handle_motion` in (game_engine.py)(game/game_engine.py) took `event.pos` -- a single sample of where the pointer was that instant -- and passed it to `Fruit.contains_point`, a plain centre-distance check. At 60 FPS a fast swipe reports positions hundreds of pixels apart, so both samples sit outside the fruit's 28px circle while the line between them goes straight through the middle. Nothing ever tests that line, so nothing slices. A slow move produces samples close enough together that one lands inside the circle, which is why slow always works.
```

The fix

```
**[game/fruit.py](game/fruit.py)** -- added `intersects_segment(x1, y1, x2, y2)`: projects the fruit's centre onto the swept segment, clamps the projection to the segment's ends, and compares that closest distance to the radius. Clamping matters - without it a swipe that *points at* the fruit but stops short would register, since the infinite line still passes through it. `contains_point` stays and is used for the degenerate zero-length case.
```

```
**[game/game_engine.py](game/game_engine.py)** -- the engine now keeps `self._last_pos` and tests the segment from the previous sample to the current one, rather than the current point alone.
```

One edge case that falls out of this: if the pointer leaves the window and re-enters elsewhere, joining those two samples would sweep a blade clear across the screen and slice everything on the way. `handle\_event` now drops the stored position on `WINDOWLEAVE`/`WINDOWFOCUSLOST` (`\_break\_blade`), which also stops the drawn trail jumping.

Testing it

```
cd Lab-4/code && python tests/test_slice_detection.py
```

tests/test_slice_detection.py runs headless -- dummy SDL video/audio drivers, no window, no real mouse. Each check parks a fruit at (350, 300) and feeds the engine synthetic `MOUSEMOTION` events through the real `handle\_event` path, so it exercises the actual wiring rather than the geometry in isolation. 12 checks, all passing: fast straddling swipe, diagonal, edge graze, plus the negatives that stop this becoming over-eager -- a swipe that misses by 5px, a swipe that stops 100px short, no double-scoring, and the slow-move case that always worked.

Two verifications beyond "the tests are green":

- Re-running the same suite with `intersects\_segment` monkeypatched back to the old point-only behaviour fails 6 of the 12 -so they genuinely pin the bug rather than just describing current behaviour.
- A 900-frame headless run of the real loop, identical spawn seed and identical fast swipes: **35 fruit sliced with the swept blade vs 3 with the old pointwise check.**

Worth knowing for your *before/after* videos: the segment is tested against the fruit's position at that frame, not against the fruit's own motion during the frame. For a fruit moving ~14px/frame against a swipe of several hundred, that's noise -- but it's the reason this is "reliable" rather than mathematically exact.

I haven't touched `Lab-4/README.md` -- the task-1 checkbox and the "bug, reproduced" section still describe the pre-fix state, so update those when you record the video. No commit made either; `PROMPTS.md` has the message you planned for this one.

PROMPT 2

Nice, that works now. Next thing.

At the moment when you hit a bomb or run out of lives, the game window just sits there frozen and prints "Game over" into the terminal, which is useless. I want a real game over screen inside the window showing the final score, with the game still visible but dimmed behind it.

Important bit -- it mustn't lock up. I should still be able to close the window with the X button, and in the next step I'm going to add a restart option so it needs to keep listening for key presses. </cross-session-message>

RESPONSE 2

Task 2 -- game-over screen. Let me look at the current state of the three files before changing them.

Now the tests. I'll factor the shared headless setup out so both suites can use it:

My off-by-one in the test's expected frame count, not in the engine -- Esc is pressed during iteration 61 (`frame == 60`) and breaks in that same pass:

Now let me actually look at the screen rather than trust pixel assertions -- rendering a real game-over frame to PNG:

The overlay is too heavy -- the scene behind is nearly invisible, which isn't "dimmed but still visible". Let me lighten it:

Text legibility against randomly-placed fruit is marginal -- adding a soft panel behind the text block while leaving the rest of the scene showing:

The panel came out *lighter* than the surround -- `draw.rect` replaces pixels on an `SRCALPHA` surface rather than blending, so its alpha is the total, not an addition. Making it a stronger black:

Done -- Task 2 is in. The loop was never actually frozen, by the way: `update()` already returned early on `game\_over` and the main loop kept turning. What was missing was anything drawn for that state, so you got a live-but-static window plus a terminal print.

What changed

```
**[game/game_engine.py](game/game_engine.py)**
```

- `\_render\_game\_over` dims the whole frame with a translucent black `SRCALPHA` overlay rather than clearing it, so the last moment of play stays visible underneath, then draws **GAME OVER**, why it ended, the final score, and the key hint. - `\_end\_game(reason)` replaces the two bare `self.game\_over = True` sites, so a bomb and running out of lives give different text ("You sliced a bomb" / "You ran out of lives"). The `print` and the `\_game\_over\_logged` flag are gone.
- `\_handle\_motion` now returns early when `game\_over`. Without this the blade kept slicing behind the overlay and the score climbed while "Final Score" sat on screen.
- `\_handle\_keydown` handles Esc/Q -> `should\_quit`. Everything is still a normal frame -- no `pygame.event.wait()`, no inner

loop -- which is what keeps the X button working and leaves the key path open for your difficulty choice. I deliberately left Space/E/M/H/R unbound for that, and there's a test asserting they stay unbound so Task 3 doesn't collide with this.

```
**[main.py](main.py)** -- checks `engine.should_quit` after render and drops out of the loop.
```

Two visual adjustments I made after rendering frames and looking at them: the first dim at alpha 180 left the scene essentially invisible, so it's 115 now; and since a fruit can land right behind the title, the text block sits on a darker rounded panel. A note in the code records why that panel's alpha is 200 rather than a small top-up -- `draw.rect` on an `SRCALPHA` surface replaces pixels instead of blending, so my first value made the panel *lighter* than its surroundings.

Tests

```
cd Lab-4/code && python tests/run_all.py
```

26 checks, all passing. I refactored the headless setup into `tests/support.py`, so
tests/test_slice_detection.py (12, unchanged in substance) and the new
tests/test_game_over.py (14) share it; each file still runs standalone.

The game-over suite asserts state (both end conditions and their reasons, play frozen, score can't climb after the end), the screen itself (title/reason/score strings, pixels dimmed but not blacked out, fruit still distinguishable from background through the dim, nothing drawn while playing), and input (Esc and Q quit, other keys don't, Esc during play does nothing, and a 120-frame stand-in loop that would hang rather than fail if anything blocked).

Beyond the unit tests I ran the real ``main()`` headless twice: it exits cleanly on a QUIT event, and exits via Esc from the game-over screen.

Here's the screen over a representative scene:

No commit made, per the same pattern as Task 1 -- ``PROMPTS.md`` has ``Lab 4 Task 2: add game over screen`` ready when you want it.

PROMPT 3

Now I want to be able to start a new game after dying instead of having to close and reopen it. On the game over screen let me pick easy, medium or hard, or quit out.

Harder should mean fruit coming faster and more bombs, easier the opposite -- pick values that actually feel different and tell me what you went with so I can tweak them.

One thing to watch: make sure starting a new game properly wipes everything from the last one. I don't want the old score, leftover fruit or the game over state hanging around. </cross-session-message>

RESPONSE 3

Task 3 -- replay with difficulty. Let me re-read the engine as it now stands before patching.

The heredoc tripped bash's parser; writing the file directly instead:

Now the two Task-2 tests whose assumptions Task 3 changed -- E/M/H are no longer inert, and the HUD now draws a centred label:

Now a look at the new screen and an end-to-end run through the real loop:

Making the key letters read as keys rather than stray capitals:

The H never arrived -- ``set_timer`` is keyed by event `*type*`, so my second call replaced the first timer. Both were KEYDOWN. Driving it from a thread instead:

The loop is still running -- and that's correct behaviour, not a bug: Esc only quits from the game-over screen, so posting it 0.2s after a replay started did nothing. Let me stop that run and redo it properly.

Task 3 is in. 45 checks passing.

Difficulty values

They live in one ``DIFFICULTIES`` dict at the top of `[game/game_engine.py](game/game_engine.py:19)`, so they're easy to tweak. Measured over a simulated 60 seconds of play:

Level	Throws / 10s	Bombs / 10s	Lives	`spawn_interval`	`bomb_chance`	`speed_scale`
--- --- --- --- ---						
Easy	8.0	1.3	5	75	0.08	0.90
Medium	12.0	2.3	3	50	0.15	1.00
Hard	20.0	4.5	2	30	0.25	1.15

Medium is close to the original starter values (55 frames / 0.15 / 3 lives) so it reads as "the normal game". Hard is 2.5x the throw rate of easy and 3.5x the bombs, which is the gap you actually feel; lives (5/3/2) is the other big lever. ``speed_scale`` is deliberately the mildest dial -- it's double-edged, since a harder throw means the fruit crosses the screen faster but also peaks higher (y?233 on hard vs y?387 on easy) and hangs around longer. If you want hard to bite more, drop ``spawn_interval`` further before touching ``speed_scale``.

Wiping the last game

The reset is structural rather than a list of assignments at the end: `**all**` per-game state now lives in ``start_game(difficulty)`` and nowhere else. ``__init__`` keeps only the fonts and ``should_quit``, then calls it. A replay and a brand-new engine therefore run identical code.

There's a test that enforces this by comparing `vars()` of a replayed engine against a freshly constructed one, so if a later task adds per-game state to `\_\_init\_\_` and forgets `start\_game`, it fails -- that's the failure mode you were worried about, caught automatically rather than by my remembering. Alongside it, individual checks for score, fruit list, game-over flag and reason, lives, spawn timer, and the blade position (a swipe from before the restart must not slice after it).

Keys: **E / M / H** replay, **Esc / Q** quit, all ignored while a game is in progress so a stray H can't wipe a run you're mid-way through. Esc deliberately does *nothing* during play -- it only quits from the game-over screen, which is what you asked for. I tripped over this myself while testing end-to-end and it's worth knowing for the video: once you replay, the only ways out are dying again or the X button. Say the word if you'd rather Esc quit at any time.

tests/test_replay.py adds 19 checks. The difficulty presets are verified by *playing* them, not just reading the dict -- hard throws >2x the fruit of easy and >1.5x the bomb share over 6000 simulated frames -- plus an ordering check so the three levels stay genuinely different if you retune them. I also updated two Task-2 tests whose assumptions this changed: E/M/H are no longer inert keys, and the HUD now draws a centred difficulty label.

Verified end-to-end through the real `main()` loop, not just units: pressing **H** on the game-over screen gives `difficulty=hard, score=0, lives=2, spawn\_interval=30, game\_over=False` with fruit flying 1.1s later, and the X button still closes the window. `cd Lab-4/code && python tests/run_all.py`

PROMPT 4

Last feature -- sound. I want a noise when you slice a fruit, a different one when you hit a bomb, and something for the game over moment.

Two catches. I don't have any sound files and I'd rather not go downloading any, so can you generate the sounds in code instead? And my laptop's audio is flaky, so if sound fails to start up the game should just carry on silently rather than crashing. Also make sure the game over sound only plays once -- I don't want it going off every frame while the screen is up.

Once that's in, that's everything I wanted -- so have a look over the whole lot and tell me whether any of the later changes broke something we did earlier, whether there's leftover code sitting around that isn't used any more, and anything that looks sloppy or could be done better. Then give me the final version of all three files. </cross-session-message>

RESPONSE 4

Task 4 -- sound. First, what's available in this environment:

numpy is here but `requirements.txt` only lists pygame, so I'll synthesise with stdlib `array` -- no new dependency on your laptop.

One test needs updating: `sounds` is a collaborator, not per-game state, so two engines legitimately hold different board objects:

Now the sound test suite:

61 passing. Now let me check the synthesised audio is actually what I intended, and export WAVs you can listen to:

Now the review pass you asked for. Checking for dead code and unused imports across the whole project:

Full-system regression check -- all four tasks through the real `main()` loop, with the sound board live:

Let me read the final files end-to-end for the review:

All 61 checks pass together, and I drove the real `main()` loop end-to-end: fast swipes scored, bomb ended it, **E** replayed into Easy (score 0, lives 5), window closed cleanly, sound board live throughout.

Sound

Generated with stdlib `array` + `math` -- no files, no numpy, nothing added to `requirements.txt`. Measured dominant frequencies from an FFT of the actual buffers:

```
| Effect | Recipe | Length | Peak freq | Vol |
|---|---|---|---|---|
| slice | 700->1500 Hz sine sweep + 25% noise | 0.10s | 770 Hz | 0.5 |
| bomb | 180->60 Hz sawtooth, noise crunch on the attack | 0.45s | 169 Hz | 0.7 |
| game over | three falling notes G4/E4/B3 + 2nd harmonic | 0.66s | 312 Hz | 0.6 |
```

Once only: the game-over sound plays in `\_end\_game` -- the state *transition* -- not in `render`. That's structurally once per game no matter how long the screen sits there, and it's why the old `\_game\_over\_logged` flag is gone. The original `print` was in `render`, which is exactly the trap you were worried about.

****Flaky audio:**** `SoundBoard` wraps startup in a broad `except`, prints one line, and sets `enabled = False`; every `play()` is then a no-op. It also goes quiet if the device vanishes mid-game (headphones unplugged) rather than raising on every slice. Tested by making `pygame.mixer.init` raise and then asserting a fruit still slices and scores.

One deliberate choice to flag: hitting a bomb plays the thud *and* the game-over jingle, overlapping. Sounds fine to me, but say the word if you want them staggered.

Review of the whole thing

****Did later tasks break earlier ones?**** Nothing in the game, but three *tests* encoded assumptions that later tasks legitimately invalidated, and I updated each rather than papering over it:

- Task 2 asserted E/M/H were inert keys -- Task 3 gave them meaning.
- Task 2 asserted nothing was drawn centred during play -- Task 3's HUD difficulty label is centred. Tightened to "no game-overtext".
- Task 3's "fresh engine == replayed engine" check broke on Task 4's `sounds` attribute. That one was the test being right and needing refinement: the sound board is a collaborator that outlives a game, so `start\_game` is correct *not* to reset it.

****Dead code removed:****

- `handle\_input()` -- a `pass` method the loop called every frame, unused since the starter. Gone, along with its call in `main.py`.
- `getattr(fruit, "color", WHITE)` in `render` -- a guard that existed only because colour was bolted onto fruit after construction. Colour is now a `Fruit` constructor argument, so both the guard and the external assignment are gone.
- `max(self.lives, 0)` in the HUD -- that was masking a real (if cosmetic) bug: lives can hit ?1 when two fruit are missed in the same frame. Clamped at the source instead.
- An unused import, and a text-capture helper duplicated across two test files.

****Things I'd flag but left alone:****

- Esc quits only from the game-over screen (your Task 3 decision), so mid-game the X button is the only way out.
- `\_render\_game\_over` uses hard-coded Y coordinates -- fine at 700x600, won't reflow if the window is ever resizable. Same for the HUD's `width - 130`.
- The project is now **four** source files, not three: I added `game/sounds.py` rather than putting 130 lines of waveform maths inside the game engine. The starter README's folder structure is out of date, as is `requirements.txt` (says `pygame`, you're running `pygame-ce`).

Final files

****`main.py`****

```
import pygame from game.game_engine
import GameEngine # Initialize
pygame/Start application pygame.init()
# Screen dimensions
WIDTH, HEIGHT = 700, 600 SCREEN =
pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Fruit Slice - Pygame Version")
# Colors
DARK_BLUE = (20, 25, 45)
# Clock
clock = pygame.time.Clock()
FPS = 60 def main():
    engine = GameEngine(WIDTH, HEIGHT)
    running = True while running:
        SCREEN.fill(DARK_BLUE) for event
        in pygame.event.get(): if
        event.type == pygame.QUIT:
            running = False
    else:
        engine.handle_event(event) engine.update()
    engine.render(SCREEN) # The game over screen is drawn as an ordinary
    frame, so the loop keeps # turning; it asks to close through this flag
    rather than blocking.
    if engine.should_quit:
        running = False
    pygame.display.flip()
    clock.tick(FPS)
    pygame.quit() if __name__ ==
    "__main__": main()
```

****`game/fruit.py`****

```
import math
class Fruit:
```

```

    def __init__(self, x, y, vx, vy, gravity, radius=28, kind="fruit",
color=(255, 255, 255)):
        self.x = x            self.y = y
self.vx = vx                self.vy = vy
self.gravity = gravity      self.radius =
radius                      self.kind = kind # "fruit" or
"bomb"                      self.color = color
self.sliced = False        def update(self):
        self.vy += self.gravity
self.x += self.vx          self.y +=
self.vy                    def
contains_point(self, x, y):
        return math.hypot(self.x - x, self.y - y) <= self.radius    def
intersects_segment(self, x1, y1, x2, y2):          # A reported mouse position
is only a sample of where the blade was at
        # one instant. Testing the whole segment the blade swept between two
        # samples is what makes a fast swipe register: the two endpoints can sit
        # either side of the fruit with neither one inside the circle, yet the
# line between them still cuts through it.
        dx = x2 - x1        dy
= y2 - y1                if dx == 0
and dy == 0:
        return self.contains_point(x1, y1)          # Closest point on the
segment to the centre: project the centre onto
        # the line, then clamp to the segment so a fruit beyond either end        # isn't
counted as hit.        t = ((self.x - x1) * dx + (self.y - y1) * dy) / (dx * dx + dy *
dy)        t = max(0.0, min(1.0, t))        return math.hypot(self.x - (x1 + t * dx),
self.y - (y1 + t * dy)) <= self.radius    def off_screen(self, height):        return
self.y - self.radius > height    **game/sounds.py** (new)

"""Sound effects, generated in code - there are no audio files to ship.
Nothing in here is allowed to take the game down with it. If the mixer will
not start (no device, drivers playing up, audio already in use) the board
reports itself disabled and every play() is a no-op, so the game runs silently
instead of crashing. """ import array import math import random import pygame
SAMPLE_RATE = 44100
PEAK = 32767 # loudest signed 16-bit sample
# Its own generator, so making the noise in these effects cannot shift the
# sequence the game uses to throw fruit. _noise = random.Random(1234) def
_envelope(i, n, attack_frac=0.02, decay=5.0):        """Quick fade in,
exponential fade out - stops clicks at both ends."""        attack = max(1,
int(n * attack_frac))        if i < attack:
        return i / attack
        return math.exp(-decay * (i - attack) / max(1, n - attack)) def
_swish(duration=0.10, f0=700.0, f1=1500.0, noise=0.25):        """Short
bright rising sweep - the blade going through a fruit."""        n =
int(SAMPLE_RATE * duration)        frames = []        phase = 0.0        for i
in range(n):
        t = i / n
        phase += 2 * math.pi * (f0 + (f1 - f0) * t) / SAMPLE_RATE
value = math.sin(phase) * (1 - noise) + _noise.uniform(-1, 1) * noise
frames.append(value * _envelope(i, n))        return frames def
_thud(duration=0.45, f0=180.0, f1=60.0):        """Low, buzzy and falling, with
grit on the front - hitting a bomb."""        n = int(SAMPLE_RATE * duration)
frames = []        phase = 0.0        for i in range(n):        t = i / n
        phase += 2 * math.pi * (f0 + (f1 - f0) * t) / SAMPLE_RATE
saw = 2 * ((phase / (2 * math.pi)) % 1.0) - 1        grit =
_noise.uniform(-1, 1) * 0.35 * math.exp(-12 * t)
frames.append((saw * 0.7 + grit) * _envelope(i, n, decay=3.0))
return frames def _fall(notes=(392.0, 311.1, 233.1),
note_duration=0.22):        """Three notes walking downwards - the game
over moment."""        frames = []        for freq in notes:        n =
int(SAMPLE_RATE * note_duration)        phase = 0.0        for i in
range(n):
        phase += 2 * math.pi * freq / SAMPLE_RATE        # A
little second harmonic, so it is not a bare sine tone.
value = math.sin(phase) * 0.8 + math.sin(2 * phase) * 0.2
frames.append(value * _envelope(i, n, decay=4.0))        return frames
def _to_sound(frames, channels):        """Turn floats in [-1, 1] into a
Sound the open mixer can play."""        samples = array.array("h")
for value in frames:
        sample = int(max(-1.0, min(1.0, value)) * PEAK)        for _ in
range(channels): # the same signal in every channel
samples.append(sample)        return pygame.mixer.Sound(buffer=samples.tobytes())
class SoundBoard:        """The game's three effects, built once at startup.
`enabled` says whether any noise will actually come out; the game does not
need to care either way.        """
    RECIPES = {
        "slice": (_swish, 0.5),
        "bomb": (_thud, 0.7),
        "game_over": (_fall, 0.6),
    }
    def __init__(self):        self.enabled = False
self._sounds = {}        self._build()    def _build(self):        try:
# pygame.init() may already have opened the mixer in a format that
# does not match what is generated here, so take it over.        if
pygame.mixer.get_init():

```

```

pygame.mixer.quit()
pygame.mixer.init(frequency=SAMPLE_RATE, size=-16, channels=1)
opened = pygame.mixer.get_init()          if not opened or opened[1] !=
-16:
    raise pygame.error(f"unusable mixer format: {opened}")
channels = opened[2]
for name, (recipe, volume) in
self.RECIPES.items():
    sound = _to_sound(recipe(), channels)
sound.set_volume(volume)                self._sounds[name] = sound
self.enabled = True                      except Exception as exc: # audio failures come in
many shapes                             # Deliberately broad: a missing device, a driver that
refuses to                             # open, a format the build cannot do. None of it is
worth a crash.
    print(f"Sound unavailable ({exc}) - playing without it.")
self.enabled = False                    self._sounds = {}    def play(self,
name):
    if not self.enabled:
        return
    sound =
self._sounds.get(name)                  if
sound is None:
    return
try:
    sound.play()                        except pygame.error:
# Device
went away mid-game (headphones unplugged, say). Go quiet    # rather
than throwing on every slice from here on.                self.enabled = False

```

``game/game\_engine.py`` -- 225 lines, printed in full in the tool output above; the Task 4 changes to it are just the `SoundBoard` import, `sounds=None` constructor parameter, the two `sounds.play(...)` calls in `_slice`, one in `_end_game`, plus the cleanups listed above. Grab it from `[game/game_engine.py](game/game_engine.py)` rather than re-reading it here. `cd Lab-4/code && python tests/run_all.py`